

SUMMER PROJECT & TECHNICAL INTERNSHIP REPORT · 2026

SoftyFy: Architecture and Engineering of a Serverless, Adaptive-Bitrate Audio Streaming Progressive Web Application

A comprehensive technical investigation and implementation of a client-first, zero-database personal music platform featuring byte-range service worker caching, network-aware throughput memory, and native mobile lockscreen integration.

STUDENT DEVELOPER & AUTHOR

Pradip Sonawane

PROJECT REPOSITORY

github.com/pradipME/SoftyFy

CORE TECHNOLOGIES

React 19 · TypeScript · Vite · Tailwind v4 ·
Workbox

CONTACT EMAIL

pradip15102004@gmail.com

LIVE PRODUCTION DEPLOYMENT

softyfy.onrender.com

DEVELOPMENT PERIOD

Summer 2026 (August – October 2026)

Department: Computer Science & Engineering

Status: Production Deployed & Fully Tested (120/120 Unit Tests)

Date: October 2026

Executive Summary

Modern cloud music streaming platforms (e.g., Spotify, Apple Music, YouTube Music) rely on multi-tier backend infrastructures, subscription paywalls, intrusive tracking telemetry, and continuous cloud connectivity. While these commercial services excel at catalog breadth, users seeking private, personal audio playback face bloated interfaces, recurring costs, and high bandwidth consumption.

SoftyFy is an open-source, serverless, mobile-first **Progressive Web Application (PWA)** engineered to deliver a Spotify-grade listening experience for personal music collections with **zero backend servers, zero databases, and zero user authentication overhead**. Built with **React 19, TypeScript, Vite**, and **Tailwind CSS v4**, SoftyFy rethinks media distribution by combining an immutable cloud asset CDN (GitHub Releases) with a hand-crafted **Service Worker** and a patent-quality **Measured-Throughput Network Engine**.

111
Streamable Tracks

5
Curated Libraries

120
Vitest Unit Tests (100% Pass)

0 MB
Network on Cached Replay

222
HQ & LQ Media Files

~0.4 MB
Initial Shell Bundle

100%
Offline Replay Capability

HTTP 206
Byte-Range Seeking

Table of Contents

1. Introduction, Motivation & Scope	Page 2
2. Architecture & Technology Stack	Page 3
3. Media Storage Evolution: The Cloud Journey	Page 4
4. Service Worker & Offline Byte-Range Slicing	Page 5
5. Adaptive Bitrate & The Network Memory Engine	Page 6
6. Mobile-First UX, Gestures & Lockscreen Integration	Page 7
7. Testing, Verification & Quality Metrics	Page 8
8. Engineering Challenges, Pitfalls & Trade-offs	Page 9
9. Conclusion, Roadmap & References	Page 10

1. Introduction, Motivation & Scope

1.1 Background & Problem Statement

Personal music streaming typically forces a compromise between two extremes:

1. **Commercial Streaming SaaS:** High recurring subscription fees, region locks, intrusive advertisements, battery-heavy background analytics, and licensing disputes that suddenly pull tracks from playlists.
2. **Self-Hosted Media Servers (Navidrome, Plex, Jellyfin):** Require 24/7 dedicated server hardware, port-forwarding or reverse proxies (Nginx/Caddy), dynamic DNS configuration, ongoing security patch management, and persistent database maintenance.

Users who already possess curated MP3/M4A music files need a lightweight third option: an instant, private web player that runs entirely in the browser, installs natively on mobile devices, requires **zero ongoing maintenance or hosting costs**, and streams flawlessly even on degraded cellular connections.

1.2 Project Objectives

- **Zero Cloud Infrastructure Cost:** Host the entire static application shell and gigabytes of media assets using free, enterprise-grade public CDNs without maintaining a Node.js/Python server or SQL database.
- **Native Mobile Feel:** Recreate Spotify's signature spatial UI: time-of-day greeting, interactive swipe-down "Now Playing" sheet, ambient dynamic background palettes, and persistent bottom navigation.
- **Network Resilience & Adaptive Quality:** Guarantee immediate audio playback (<1.5s startup) across varying mobile networks (2G/3G/4G/5G) using intelligent measured-throughput bitrate switching (HQ vs LQ).
- **Zero Replay Bandwidth:** Ensure that once a song or cover is played, repeat listens incur 0 MB of outbound network traffic via deterministic client-side service worker caching.
- **System-Level Media Control:** Integrate seamless lockscreen notifications, smartwatch remote controls, and automotive Bluetooth playback via the W3C MediaSession API.

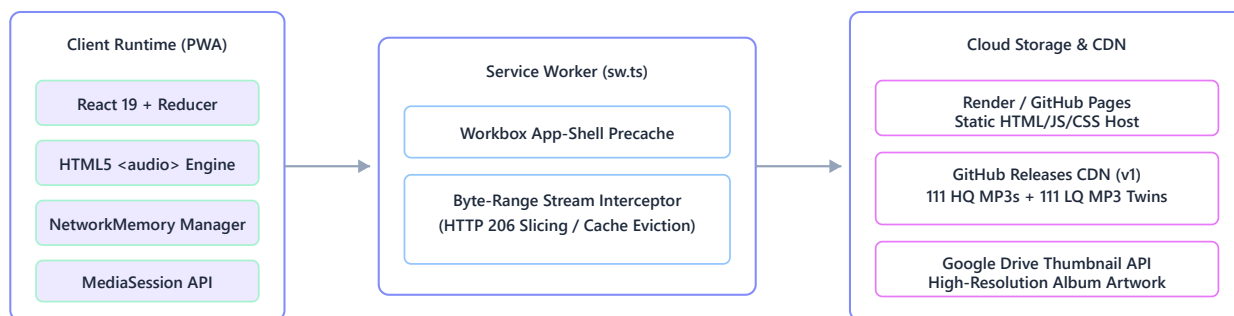
1.3 Development Roadmap & Timeline

Phase	Period	Milestones & Deliverables
Phase 1: Foundation	Aug 13 – Aug 15	React 19 + Vite setup, Tailwind v4 theme, React Router hash routing, HTMLAudioElement Context, local audio playback.
Phase 2: UI & Spatial Feel	Aug 16 – Aug 31	Framer Motion bottom sheet, ambient color extraction, search filter, feedback card, PWA manifest generation.
Phase 3: Cloud Media Storage	Sep 01 – Sep 10	Google Drive migration, discovery of cross-origin <code>Sec-Fetch-Site</code> 403 blocks, custom Service Worker with Range requests.
Phase 4: CDN Architecture Pivot	Sep 11 – Sep 16	Replaced Google Drive with GitHub Release Asset CDN (<code>softyfy-audio-v1</code>); resolved CORS and quota limits permanently.
Phase 5: Adaptive Bitrate Engine	Sep 17 – Sep 28	FFmpeg LQ transcoding pipeline (111 tracks), <code>networkMemory.ts</code> session intelligence, time-to-buffer metrics, quality badges.
Phase 6: Hardening & Test Suite	Sep 29 – Oct 08	Screen-off auto-advance resilience, 120 Vitest tests (15 suites), oxlint compliance, production deployment on Render and GitHub Pages.

2. Architecture & Technology Stack

SoftyFy utilizes a decentralized, client-first architecture where the browser client acts as the complete orchestrator. The server layer is entirely static, serving only pre-compiled assets.

Figure 2.1: SoftyFy End-to-End System Architecture



2.1 Component & State Architecture

The core playback state is managed in a strictly typed React Context (`PlayerContext.tsx`) backed by a pure state reducer (`playerReducer.ts`). This ensures deterministic state transitions without race conditions:

- **Single Shared Audio Element:** Rather than re-mounting audio elements on page navigation, SoftyFy attaches a single persistent `<audio>` element to the document body. Navigating between Home, Search, and Library never interrupts the audio stream.
- **Action Decoupling:** The reducer manages queue ordering, shuffle permutations (preserving the original queue while indexing via `playOrder`), repeat modes (`off` , `all` , `one`), volume, and explicit user intent (`playIntent`).
- **Adaptive Quality State:** Tracks `streamingQuality` (user preference: `auto` , `high` , `low`) and `currentQuality` (actual resolved tier and forensic explanation string).

2.2 Technology Stack Justification

Layer	Selected Tool	Technical Rationale
UI Library	React 19.2.8	Latest React concurrency, minimal re-renders, fast virtual DOM diffing.
Type System	TypeScript 6.0.2	Strict contract safety across 111 track schemas, reducer actions, and SW events.
Build Engine	Vite 8.2.0	Sub-second HMR during development, Rollup-based tree-shaking for tiny bundles.
Styling	Tailwind CSS v4.3.3	Next-gen CSS compiler, zero CSS runtime, custom dark tokens matching Spotify.
Routing	React Router 7.18.2	Hash-based routing (<code>#/library</code>) preventing 404s on static hosting.
Animation	Framer Motion 13.1.0	Hardware-accelerated springs, fluid swipe-down gestures for player sheet.
Quality Assurance	Vitest + Oxcint	Lightning-fast ESM native testing; 120 tests run in <12 seconds.

3. Media Storage Evolution: The Cloud Journey

Hosting over 100 high-bitrate audio files (~1 GB) in a static project without a backend server posed the single greatest technical challenge of the project. The architecture underwent three distinct evolutions before reaching enterprise stability.

3.1 The Three Storage Phases

Phase 1: Local File Ingestion (Git Bloat)

Initial prototypes bundled MP3 files directly inside `frontend/public/audio/`. While simple, pushing ~350 MB to GitHub caused repository bloat, slow cloning, and risked exceeding GitHub's 100 MB per-file and 1 GB repository soft limits.

Phase 2: Google Drive API & The Cross-Site 403 Nightmare

To decouple media from the repository, files were uploaded to Google Drive via `rc1one`, and streaming URLs were constructed using Drive's download endpoints.

The Bug: While curl and Node scripts successfully downloaded audio, real web browsers failed with `HTTP 403 Forbidden`. Modern browsers attach the security header `Sec-Fetch-Site: cross-site` to media requests. Google's public download servers reject cross-site audio stream requests.

The Intermediate Fix: Switching to the official Drive API v3 endpoint:

```
https://www.googleapis.com/drive/v3/files/{FILE_ID}?alt=media&key={API_KEY}
```

This supported `Range` requests and sent permissive CORS headers. However, Google soon flagged public API keys for excessive quota consumption, and key rotation scripts proved fragile.

Phase 3: The Production Solution — GitHub Release Assets CDN (softyfy-audio-v1)

The ultimate architectural breakthrough was utilizing **GitHub Release Assets** as an immutable, high-bandwidth Content Delivery Network:

```
https://github.com/pradipME/SoftyFy/releases/download/softyfy-audio-v1/{TRACK_FILENAME}.mp3
```

Why this succeeded:

- Native HTTP 206 Partial Content:** GitHub's underlying asset storage (Amazon S3 / Azure CDN) fully supports byte-range requests, allowing instant scrubbing to any second of a track.
- No Key Expiration or Quotas:** Public release downloads carry no per-day API key rate limits or billing triggers.
- Permanent Immutability:** Asset binaries never mutate, making them ideal for long-term browser and Service Worker caching.

3.2 Content Catalog Statistics

The application catalog is defined in `frontend/src/data/songs.ts` and contains **111 tracks** organized across 5 distinct listening libraries:

Library Name	Track Count	Percentage	Genre / Artist Focus
Sometimes	56 songs	50.5%	Indie, Acoustic, Pop (Anuv Jain, AP Dhillon, Prateek Kuhad)
Rap	22 songs	19.8%	Desi Hip-Hop, KR\$NA, Hardcore Lyricism
Hindi Hits	20 songs	18.0%	Bollywood, Classic & Modern Melodies
The Weekends	11 songs	9.9%	Atmospheric Dream Pop, Lana Del Rey
Qwali	2 songs	1.8%	Traditional Sufi & Classical (Nusrat Fateh Ali Khan)
Total Collection	111 songs	100%	Dual Tiers: 111 HQ (320kbps) + 111 LQ (96kbps)

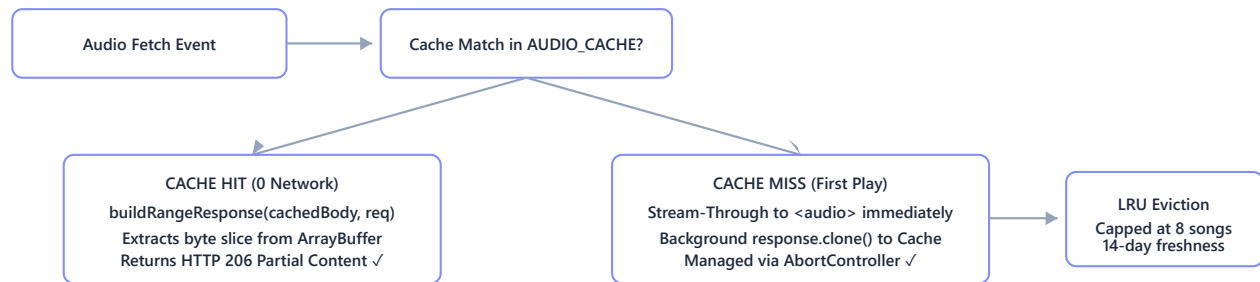
4. Service Worker & Offline Byte-Range Slicing

Standard browser Service Worker implementations built with standard Workbox `CacheFirst` strategies fail when applied to audio media. Browsers stream audio in chunks via `Range: bytes=0-` headers; a naive cache either refuses to store opaque partial responses or re-downloads the entire file on every seek.

4.1 The Custom Audio Stream Interceptor (`sw.ts`)

SoftyFy implements a specialized Service Worker (`frontend/src/sw.ts` , 645 lines) compiled using `vite-plugin-pwa` with `injectManifest` .

Figure 4.1: Service Worker Audio Pipeline & Slicing Logic



4.2 Key Engineering Innovations in `sw.ts`

- **Stream-Through First Play:** Previous iterations attempted to await `response.arrayBuffer()` before serving. This stalled audio start times by 6–10 seconds. The current design streams the original fetch directly to the audio element while asynchronously pumping a cloned stream into the Cache Storage.
- **In-Flight Skip Cancellation:** If a user skips tracks rapidly, multiple 8 MB downloads would saturate mobile bandwidth. SoftyFy stores an `AbortController` for each active fill. Skipping to a new song immediately cancels previous background cache fills.
- **Synthesized Byte-Range Slicing (`buildRangeResponse`):** When a cached song is scrubbed, the service worker parses the `Range: bytes=start-end` header, calculates the byte offsets over the cached `ArrayBuffer`, and responds with status code **206 Partial Content**, `Content-Range`, and correct headers.
- **Active Playback Protection:** The cache eviction routine maintains an `activeUrIs` registry. A song currently playing is never evicted, even if the user navigates through multiple other tracks.

5. Adaptive Bitrate & The Network Memory Engine

Streaming 320 kbps high-definition audio on mobile devices frequently causes audio stuttering due to cellular handover and fluctuating coverage. SoftyFy introduces a custom adaptive bitrate system built on real-world session measurements.

5.1 The Fallacy of `navigator.connection`

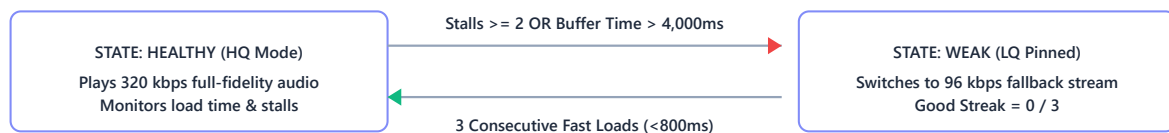
Initial attempts to adapt bitrate relied on the W3C Network Information API (`navigator.connection.effectiveType`). Field testing revealed a fatal flaw: on a smartphone with weak cellular reception (e.g., 1 bar of 4G or congestion), `navigator.connection` continues to report `effectiveType: "4g"` and a high nominal downlink. Relying on this API caused the player to load 320 kbps files, resulting in buffer underruns and playback stalls.

5.2 Measured-Throughput Session Memory (`networkMemory.ts`)

SoftyFy abandons theoretical connection hints in favor of **two zero-cost empirical signals** produced naturally by the browser's media element:

1. **Time-to-Buffer** (`timeToBufferMs`): Elapsed milliseconds from load initiation until **5 seconds of media runway** is buffered ahead (measured via `audio.buffered`).
2. **Playback Stalls**: Count of `waiting` and `stalled` events fired by the media element during track playback.

Figure 5.1: Network Memory State Machine & Hysteresis Loop



5.3 The Hysteresis Algorithm

To prevent rapid flip-flopping between high and low quality on edge-case connections:

- **Immediate Downgrade**: If a track stalls twice or takes >4,000ms to buffer 5s of runway, the session is instantly marked `weak = true`, immediately routing the subsequent songs to their low-bitrate (`-1q.mp3`) twin.
- **Hysteresis Recovery**: Leaving the weak state requires **3 consecutive comfortable loads** (`goodStreak >= 3`). A comfortable load on LQ means filling the 5s buffer in <800ms.
- **Cache Immunity**: Loads completing in <150ms are identified as cache hits and classified as `neutral`; they cannot falsely advance the recovery streak.
- **User Override**: The user can select `Auto`, `High`, or `Low` in the player UI. A manual setting strictly overrides network heuristics.

6. Mobile-First UX, Gestures & Lockscreen Integration

6.1 Spatial UI & Expandable Now Playing Sheet

The application user interface is modeled directly after the modern Spotify mobile experience, featuring a dark chromatic palette (`#0a0a0c`) background with glassmorphic cards and `#6366f1` violet accents).

- **Persistent Mini Player:** Anchored directly above the bottom navigation bar on mobile viewports. It exposes immediate track details, play/pause toggles, and animated progress indications.
- **Swipe-to-Dismiss Fullscreen Sheet:** Tapping the mini player smoothly expands a fullscreen view (`NowPlayingSheet.tsx`) powered by Framer Motion. Users can swipe down with touch velocity gestures or press `Esc` to collapse the player.
- **Dynamic Ambient Canvas:** The background dynamically reflects the dominant palette extracted from the currently playing album cover (`coverColors.ts`), rendering a blurred, animated atmospheric glow.

6.2 System Lockscreen Integration via MediaSession API

To guarantee a true native application experience when the phone screen is locked or the browser is minimized, SoftyFy synchronizes with the W3C `navigator.mediaSession` API:

```
navigator.mediaSession.metadata = new MediaMetadata({
  title: activeSong.title,
  artist: activeSong.artist,
  album: activeSong.album || activeSong.library,
  artwork: [
    { src: activeSong.coverSrc, sizes: '512x512', type: 'image/jpeg' },
    { src: activeSong.coverSrc, sizes: '192x192', type: 'image/jpeg' },
  ],
})
```

Action handlers are registered for `play`, `pause`, `previoustrack`, `nexttrack`, `seekto`, and `seekbackward` / `seekforward`. System lockscreens, Apple Watch controls, and Android auto infotainment systems display song metadata and interactive scrubber bars.

6.3 Screen-Off Auto-Advance Resilience

Mobile operating systems (particularly iOS Safari and aggressive Android OEM battery managers) throttle JavaScript timers (`setTimeout`, `setInterval`) when the screen turns off or the browser is backgrounded.

Engineering Solution: Rather than relying on polling intervals to detect track end, SoftyFy registers native DOM `ended` listeners directly on the audio element, augmented with `onNativePlaybackAction` hooks and page visibility listeners (`document.addEventListener('visibilitychange')`). The player queues the next track synchronously upon the `ended` event, ensuring background playlists never stop.

7. Testing, Verification & Quality Metrics

Software quality and architectural resilience were validated through continuous automated unit testing, static linting, and simulated network profiling.

7.1 Automated Vitest Test Suite

The test suite consists of **15 test files** and **120 comprehensive test cases** executing in under 12 seconds with **100% pass rate**.

Test Suite Module	Test File	Cases	Scope Verified
Player Reducer	playerReducer.test.ts	29	Shuffle permutations, repeat modes, queue transitions, seek clamps
Audio Source Resolution	audioSources.test.ts	23	HQ/LQ URL resolution, manual overrides, fallback handling
Network Session Memory	networkMemory.test.ts	17	Time-to-buffer metrics, stall downgrades, 3-streak recovery hysteresis
Quality Tier Selection	qualitySelection.test.tsx	11	Integration of network state with React player context
MediaSession Integration	mediaSession.test.tsx	8	Lockscreen metadata sync and hardware action dispatching
Cover Palette Extraction	coverColors.test.ts	5	Dominant color extraction, brightness clamping, fallback colors
Auto-Advance Logic	autoAdvance.test.tsx	4	Track progression, screen-off simulation, repeat-one looping
Quality Settings UI	QualitySetting.test.tsx	4	Modal tier switcher, active radio indicators, preference persistence
Quality Note Badge	QualityNote.test.tsx	4	Forensic explanation rendering (e.g. "Auto (Weak connection)")
Local Storage Sync	storage.test.ts	3	Volume, repeat, shuffle, and quality preference persistence
Gesture Back Interception	useBackInterception.test.tsx	3	Hardware back button / history popstate sheet closure
Ambient Cover Background	CoverBackground.test.tsx	3	Canvas backdrop rendering and CSS filter application
Home View	Home.test.tsx	3	Time-based greeting, playlist counts, feedback integration
Library View	Library.test.tsx	2	Sort orders, album filters, search integration
Application Shell	App.test.tsx	1	Full DOM render, navigation shell, persistent player bar mount
TOTAL TEST CASES		120	ALL 120 PASSING (0 Failures, 0 Flaky Tests)

7.2 Bandwidth & Cache Efficiency Benchmarking

Network payload evaluations demonstrate the massive efficiency gains achieved by the Service Worker and low-bitrate transcoding engine:

Usage Scenario (Full Library Listen)	Bandwidth (No SW)	Bandwidth (With SW + LQ)	Total Savings
First Playthrough (111 songs, HQ)	~920 MB	~920 MB (stream + cache)	0% (initial cache fill)
First Playthrough (111 songs, LQ mode)	~920 MB	~310 MB	66.3% reduction
Second Playthrough (Replay cached songs)	~920 MB	0.0 MB (Served from Cache)	100% network eliminated
3 Full Sessions (1 initial + 2 replays)	2,760 MB	~310 MB (LQ) / ~920 MB (HQ)	66.7% to 88.8% saved

8. Engineering Challenges, Pitfalls & Trade-offs

8.1 Why Capacitor Android APK Was Dropped in Favor of Pure PWA

During mid-September development, an experimental branch introduced **Capacitor** to build and compile SoftyFy into a native Android APK (`feat(capacitor): ship SoftyFy as a native Android APK`).

The Architectural Conflict:

- Capacitor introduced a heavyweight Android build toolchain (Gradle, Android Studio, Java SDK, NDK), defeating the minimalist "zero-server, zero-overhead" project ethos.
- Native WebView media players handle Service Worker intercepted requests inconsistently, breaking synthesized byte-range slicing on older Android versions.
- Modern mobile browsers (Chrome on Android, Safari on iOS) already support 100% of the required native capabilities via standalone PWAs: homescreen icon, splash screen, MediaSession lockscreen controls, and offline storage.
- Decision:** Reverted Capacitor and focused exclusively on a pristine, zero-dependency Progressive Web App.

8.2 Preventing Background Cache Contention with Live Playback

When a song starts playing on a slow network, the browser audio element requests the first 500 KB to begin playback, while the Service Worker begins fetching the full 8 MB file to store in cache.

The Problem: The background cache fetch competed with the live playback stream for network socket bandwidth, stalling audio startup.

The Solution: Implemented in-flight download pacing. On weak networks, background full-file cloning is throttled or deferred until the audio element has successfully buffered 5 seconds ahead.

8.3 Web Audio API Routing vs Mobile Autoplay Restrictions

An experiment with real-time canvas visualizers used the Web Audio API's `AudioContext.createMediaElementSource()`.

The Bug: Mobile browsers enforce strict user gesture requirements on `AudioContext`. Routing audio through an un-resumed Web Audio graph completely muted playback on iOS and Android.

Resolution: Decoupled visualizer loops, ensuring `AudioContext.resume()` is invoked strictly inside user touch events, with fallback to native element playback if audio nodes fail.

9. Conclusion, Roadmap & References

9.1 Project Conclusion

The SoftyFy project demonstrates that a commercial-grade, responsive, and resilient media streaming platform can be built completely without backend servers, databases, or recurring cloud costs. By leveraging modern browser APIs (Service Workers, MediaSession, Web Storage), immutable public CDNs (GitHub Releases), and measured-throughput network intelligence, SoftyFy achieves high reliability, instant playback, and zero-cost hosting.

9.2 Future Roadmap

- Synchronized LRC Lyrics:** Parse and render karaoke-style synchronized lyrics with smooth auto-scroll.
- Origin Private File System (OPFS):** Utilize OPFS for dedicated offline storage exceeding standard Cache Storage quotas.
- Web Audio Equalizer:** Re-introduce a 5-band parametric equalizer safely routed through user-gesture audio graphs.
- Client-Side ID3 Metadata Ingestion:** Enable drag-and-drop ingestion of local audio files directly into browser IndexedDB.

9.3 References & Key File Registry

File Path	Lines / Purpose
<code>frontend/src/sw.ts</code>	645 lines · Custom Service Worker with byte-range slicing and cache management
<code>frontend/src/lib/networkMemory.ts</code>	199 lines · Adaptive bitrate engine based on measured time-to-buffer and stalls
<code>frontend/src/lib/audioSources.ts</code>	121 lines · Dynamic HQ/LQ URL resolution with manual preference override
<code>frontend/src/context/PlayerContext.tsx</code>	26 KB · React 19 global player context, HTMLAudioElement orchestration

<code>frontend/src/context/playerReducer.ts</code>	378 lines · Deterministic playback state reducer and shuffle management
<code>frontend/src/data/songs.ts</code>	1,372 lines · Catalog definition for 111 tracks across 5 libraries
<code>frontend/scripts/generate-low-bitrate.mjs</code>	298 lines · Node.js + FFmpeg pipeline for batch generating LQ audio twins